Week4_Gueorgui_Poklitar

June 28, 2026

1 Week 4 - Logistic Regression and Feature Scaling

[1]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn kagglehub
```

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score, roc_auc_score, roc_curve, confusion_matrix,
    classification_report, precision_recall_curve, average_precision_score
)

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

[3]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

#Cleaning
df = df.drop_duplicates()
```

```
# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
 ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

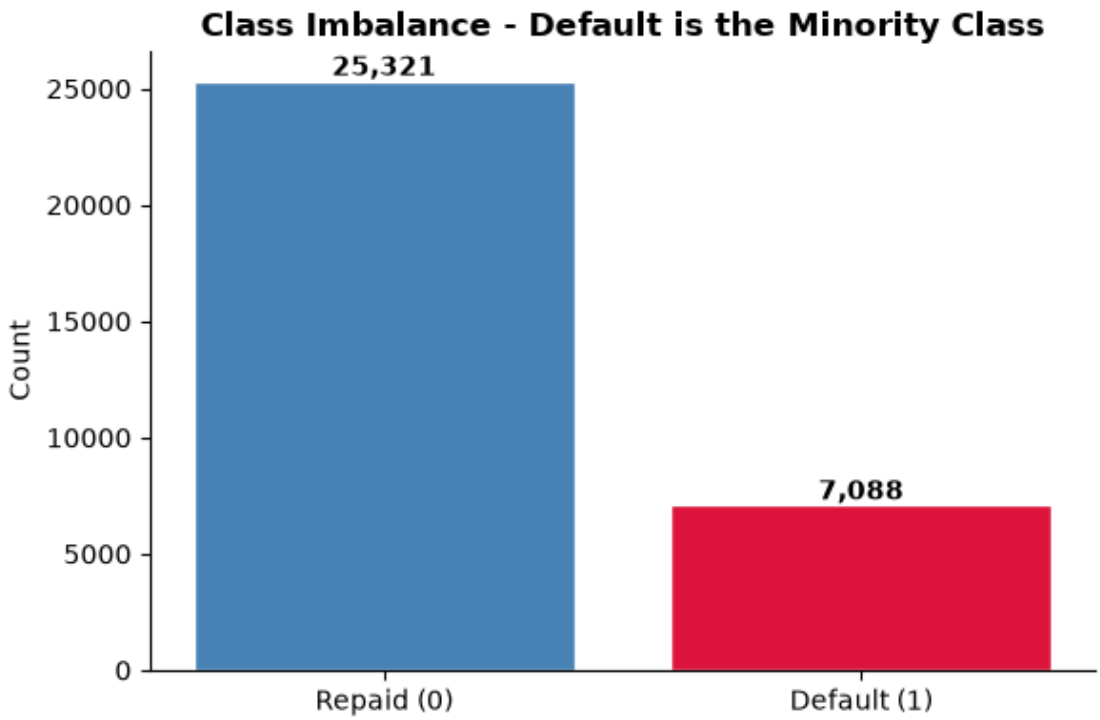
```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                    int64
person_income                 int64
person_home_ownership           str
person_emp_length           float64
loan_intent                     str
loan_grade                      str
loan_amnt                     int64
loan_int_rate               float64
loan_status                   int64
loan_percent_income         float64
cb_person_default_on_file       str
cb_person_cred_hist_length    int64
dtype: object
```

1.2 2. The Pivot to Classification: Predicting Default (loan_status)

[4]:
```
# New target: loan_status (1 = default). Note: loan_int_rate and loan_grade are
# legitimate predictors here (the lender knows them at origination), unlike in␣
 ↪Week 3.
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income', 'loan_int_rate']
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
```

```
X = df[numeric_features + categorical_features].copy()
y = df['loan_status'].copy()
mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Train size: {len(X_train):,} | Test size: {len(X_test):,}")
print(f"Default rate (train): {y_train.mean()*100:.2f}% | (test): {y_test.
 ↪mean()*100:.2f}%")

fig, ax = plt.subplots(figsize=(6, 4))
counts = y.value_counts().sort_index()
ax.bar(['Repaid (0)', 'Default (1)'], counts.values,
       color=['steelblue', 'crimson'], edgecolor='white')
for i, v in enumerate(counts.values):
    ax.text(i, v + max(counts.values)*0.01, f'{v:,}', ha='center',␣
 ↪fontweight='bold')
ax.set_ylabel('Count')
ax.set_title('Class Imbalance - Default is the Minority Class',␣
 ↪fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: With defaults a ~1-in-5 minority, raw accuracy is a trap␣
 ↪- a model")
print("that predicts 'never default' would already score ~80%. We stratify the␣
 ↪split and")
print("lean on AUC, recall, and precision-recall, which actually reward␣
 ↪catching defaulters.")
```

```
Train size: 25,927 | Test size: 6,482
Default rate (train): 21.87% | (test): 21.88%
```




```
Interpretation: With defaults a ~1-in-5 minority, raw accuracy is a trap - a
model
that predicts 'never default' would already score ~80%. We stratify the split
and
lean on AUC, recall, and precision-recall, which actually reward catching
defaulters.
```

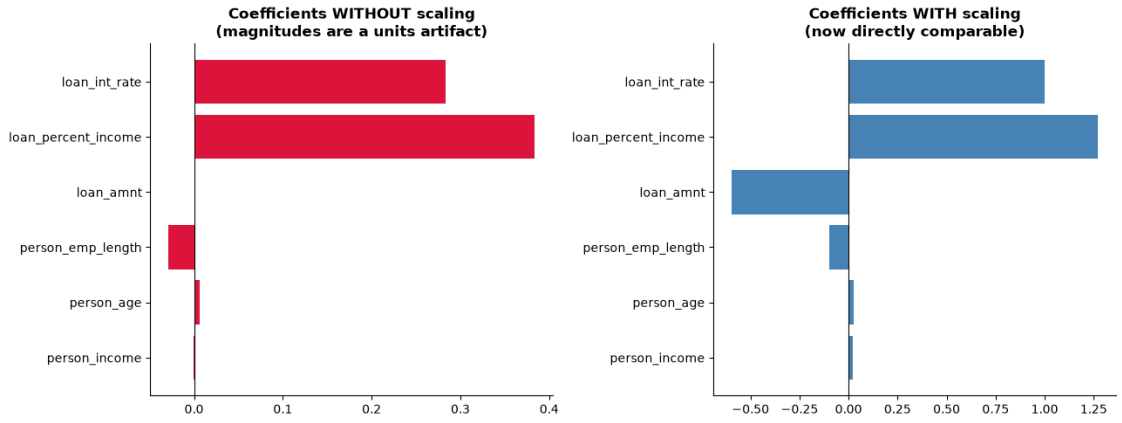
1.3 3. Feature Scaling: Why It Actually Matters

[5]:
```
# Demonstration, not assertion: logistic regression with an L2 penalty applies␣
 ↪the SAME
# shrinkage to every coefficient. Without scaling, a feature measured in tens␣
 ↪of thousands
# (person_income) and one measured in fractions (loan_percent_income) are␣
 ↪penalized on
# wildly different footings - the penalty becomes a units artifact, not a␣
 ↪modeling choice.
num_only = df[numeric_features + ['loan_status']].dropna()
Xn = num_only[numeric_features]
yn = num_only['loan_status']
Xn_tr, Xn_te, yn_tr, yn_te = train_test_split(
    Xn, yn, test_size=0.2, random_state=42, stratify=yn)
```

```
# (a) UNSCALED
clf_raw = LogisticRegression(max_iter=200, penalty='l2', C=1.0)
clf_raw.fit(Xn_tr, yn_tr)
auc_raw = roc_auc_score(yn_te, clf_raw.predict_proba(Xn_te)[:, 1])
iters_raw = int(clf_raw.n_iter_[0])

# (b) SCALED
scaler = StandardScaler()
Xn_tr_s = scaler.fit_transform(Xn_tr)
Xn_te_s = scaler.transform(Xn_te)
clf_scl = LogisticRegression(max_iter=200, penalty='l2', C=1.0)
clf_scl.fit(Xn_tr_s, yn_tr)
auc_scl = roc_auc_score(yn_te, clf_scl.predict_proba(Xn_te_s)[:, 1])
iters_scl = int(clf_scl.n_iter_[0])

print("Feature scaling: unscaled vs standardized (same model, same data)")
print(f"  Unscaled   -> AUC {auc_raw:.4f} | solver iterations: {iters_raw}")
print(f"  Standardized-> AUC {auc_scl:.4f} | solver iterations: {iters_scl}")

# Visual: how comparable are the coefficients before vs after scaling?
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].barh(numeric_features, clf_raw.coef_[0], color='crimson')
axes[0].axvline(0, color='black', linewidth=0.8)
axes[0].set_title('Coefficients WITHOUT scaling\n(magnitudes are a units␣
 ↪artifact)',
                  fontweight='bold')
axes[1].barh(numeric_features, clf_scl.coef_[0], color='steelblue')
axes[1].axvline(0, color='black', linewidth=0.8)
axes[1].set_title('Coefficients WITH scaling\n(now directly comparable)',␣
 ↪fontweight='bold')
for ax in axes:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: Scaling lets the solver converge faster and - crucially␣
 ↪- makes the")
print("L2 penalty fair across features and the coefficients comparable in size.␣
 ↪Every model")
print("from here on scales its numeric inputs inside the pipeline, with no␣
 ↪leakage (the")
print("scaler is fit on training folds only).")
```

```
Feature scaling: unscaled vs standardized (same model, same data)
  Unscaled   -> AUC 0.8158 | solver iterations: 200
```

```
Standardized-> AUC 0.8361 | solver iterations: 11
```



Interpretation: Scaling lets the solver converge faster and - crucially - makes the
L2 penalty fair across features and the coefficients comparable in size. Every model
from here on scales its numeric inputs inside the pipeline, with no leakage (the scaler is fit on training folds only).

1.4 4. Baseline Logistic Regression (Full Pipeline)

[6]:
```
# Full pipeline: scale numerics, one-hot encode categoricals, then logistic␣
 ↪regression.
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

logit_pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(max_iter=1000, class_weight='balanced'))
])
logit_pipe.fit(X_train, y_train)

proba = logit_pipe.predict_proba(X_test)[:, 1]
preds = logit_pipe.predict(X_test)
auc = roc_auc_score(y_test, proba)

print("Baseline Logistic Regression (class_weight='balanced')")
```

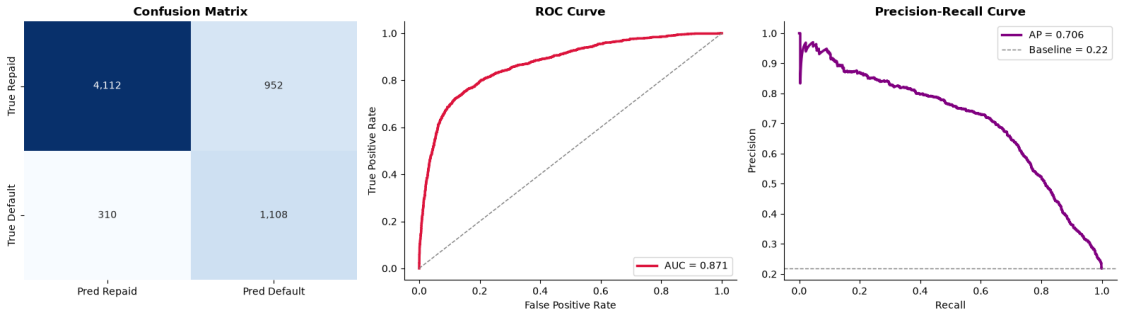
```
print(f"  Test Accuracy: {accuracy_score(y_test, preds):.4f}")
print(f"  Test AUC     : {auc:.4f}")
print()
print(classification_report(y_test, preds, target_names=['Repaid', 'Default']))
print("Note: class_weight='balanced' re-weights the minority class so the model␣
 ↪is rewarded")
print("for catching defaulters rather than coasting on the majority 'repaid'␣
 ↪class.")
```

```
Baseline Logistic Regression (class_weight='balanced')
  Test Accuracy: 0.8053
  Test AUC     : 0.8714

              precision    recall  f1-score   support

      Repaid       0.93      0.81      0.87      5064
     Default       0.54      0.78      0.64      1418

    accuracy                           0.81      6482
   macro avg       0.73      0.80      0.75      6482
weighted avg       0.84      0.81      0.82      6482

Note: class_weight='balanced' re-weights the minority class so the model is
rewarded
for catching defaulters rather than coasting on the majority 'repaid' class.
```

1.5 5. Threshold-Aware Evaluation: Confusion Matrix, ROC, Precision-Recall

[7]:
```
cm = confusion_matrix(y_test, preds)
ap = average_precision_score(y_test, proba)
fpr, tpr, _ = roc_curve(y_test, proba)
prec, rec, _ = precision_recall_curve(y_test, proba)

fig, axes = plt.subplots(1, 3, figsize=(16, 4.6))

sns.heatmap(cm, annot=True, fmt=',d', cmap='Blues', cbar=False,
            xticklabels=['Pred Repaid', 'Pred Default'],
            yticklabels=['True Repaid', 'True Default'], ax=axes[0])
axes[0].set_title('Confusion Matrix', fontweight='bold')

axes[1].plot(fpr, tpr, color='crimson', linewidth=2.5, label=f'AUC = {auc:.3f}')
axes[1].plot([0, 1], [0, 1], color='gray', linestyle='--', linewidth=1)
axes[1].set_xlabel('False Positive Rate'); axes[1].set_ylabel('True Positive␣
 ↪Rate')
axes[1].set_title('ROC Curve', fontweight='bold'); axes[1].legend(loc='lower␣
 ↪right')
```

```
axes[2].plot(rec, prec, color='purple', linewidth=2.5, label=f'AP = {ap:.3f}')
axes[2].axhline(y_test.mean(), color='gray', linestyle='--', linewidth=1,
                label=f'Baseline = {y_test.mean():.2f}')
axes[2].set_xlabel('Recall'); axes[2].set_ylabel('Precision')
axes[2].set_title('Precision-Recall Curve', fontweight='bold'); axes[2].legend()

for ax in axes[1:]:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("Interpretation: AUC reads the model's ranking quality independent of any single")
print("threshold; the precision-recall curve, anchored to the ~20% default base rate, is the")
print("more honest lens under imbalance. Where to set the cutoff is a business call - the cost")
print("of approving a defaulter vs rejecting a good borrower - not a statistical default of 0.5.")
```

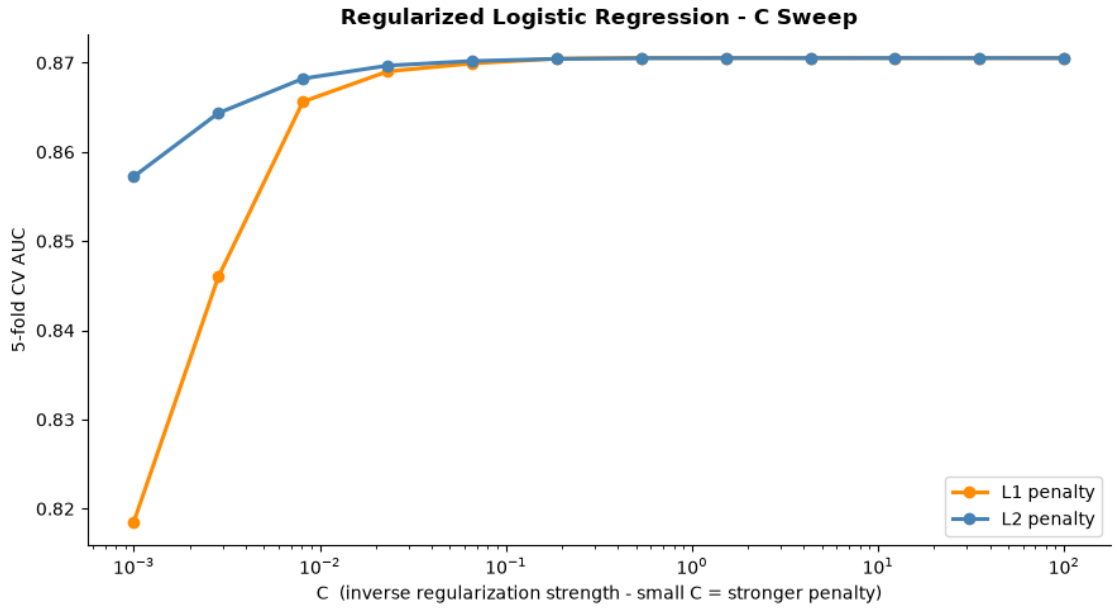



```
Interpretation: AUC reads the model's ranking quality independent of any single
threshold; the precision-recall curve, anchored to the ~20% default base rate, is the
more honest lens under imbalance. Where to set the cutoff is a business call - the cost
of approving a defaulter vs rejecting a good borrower - not a statistical default of 0.5.
```

1.6 6. Regularized Logistic Regression: The C Sweep (L1 vs L2)

[8]:
```
# C is the INVERSE of regularization strength: small C = strong penalty.
# L1 can zero features out (selection); L2 shrinks them smoothly. Same trade-offs as Week 2,
# now for classification. We sweep C and watch CV AUC for both penalties.
Cs = np.logspace(-3, 2, 12)
```

```
rows = []
for pen in ['l1', 'l2']:
    for C in Cs:
        pipe = Pipeline([
            ('prep', preprocessor),
            ('clf', LogisticRegression(penalty=pen, C=C, solver='liblinear',
                                       max_iter=2000, class_weight='balanced'))
        ])
        auc_cv = cross_val_score(pipe, X_train, y_train, cv=5,␣
 ↪scoring='roc_auc').mean()
        rows.append({'penalty': pen, 'C': C, 'CV_AUC': auc_cv})
sweep = pd.DataFrame(rows)

fig, ax = plt.subplots(figsize=(9, 5))
for pen, col in [('l1', 'darkorange'), ('l2', 'steelblue')]:
    s = sweep[sweep['penalty'] == pen]
    ax.semilogx(s['C'], s['CV_AUC'], marker='o', linewidth=2.2,
                color=col, label=f'{pen.upper()} penalty')
ax.set_xlabel('C  (inverse regularization strength - small C = stronger␣
 ↪penalty)')
ax.set_ylabel('5-fold CV AUC')
ax.set_title('Regularized Logistic Regression - C Sweep', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

best = sweep.loc[sweep['CV_AUC'].idxmax()]
print(f"Best config: {best['penalty'].upper()} penalty, C={best['C']:.4f}, "
      f"CV AUC={best['CV_AUC']:.4f}")
print()
print("Interpretation: AUC is flat across a wide band of C, which is reassuring␣
 ↪- the signal")
print("is robust and not an artifact of one lucky penalty setting. We pick a␣
 ↪moderate C: enough")
print("regularization to guard against overfitting the 30+ dummy columns,␣
 ↪without throwing away")
print("usable signal. This echoes Week 2's lesson that regularization is␣
 ↪insurance, not magic.")
```



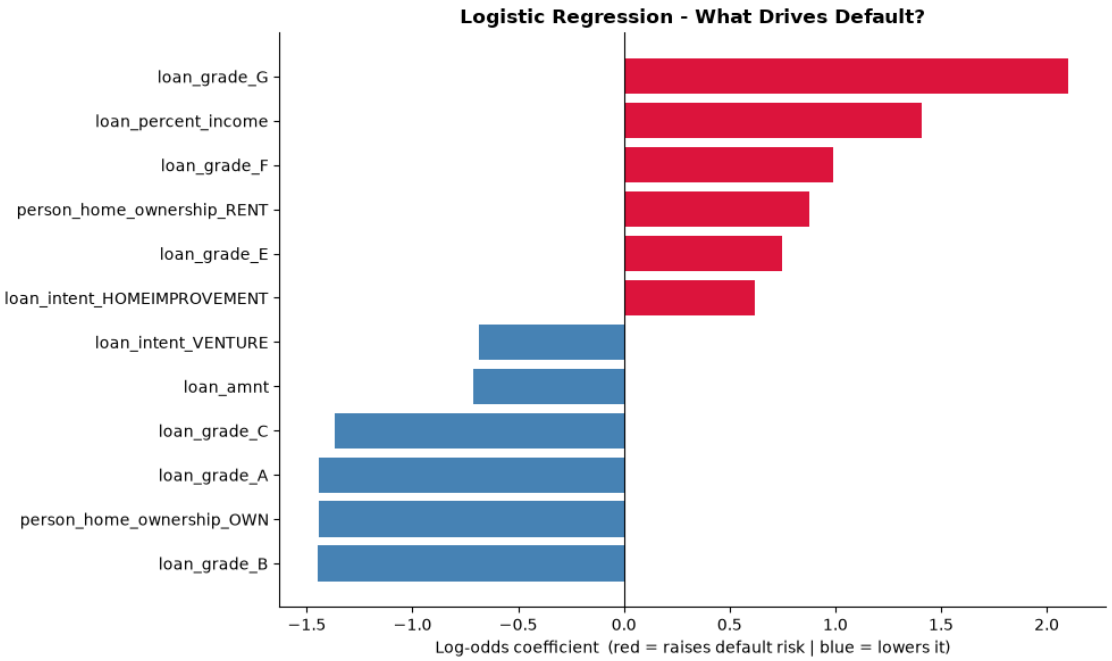

```
Best config: L1 penalty, C=0.5337, CV AUC=0.8705

Interpretation: AUC is flat across a wide band of C, which is reassuring - the
signal
is robust and not an artifact of one lucky penalty setting. We pick a moderate
C: enough
regularization to guard against overfitting the 30+ dummy columns, without
throwing away
usable signal. This echoes Week 2's lesson that regularization is insurance, not
magic.
```

1.7 7. Reading the Model: Odds Ratios a Risk Officer Can Use

[9]:
```python
# Refit a clean L2 model and translate coefficients into odds ratios.
# Odds ratio = exp(coef): >1 raises default odds, <1 lowers them, per 1-SD move
# (numerics are standardized) or per category-vs-baseline (dummies).
final = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(penalty='l2', C=float(best['C']),
                               solver='liblinear', max_iter=2000,
                               class_weight='balanced'))
]).fit(X_train, y_train)

cat_names = final.named_steps['prep'].named_transformers_['cat'] \
    .get_feature_names_out(categorical_features).tolist()
all_names = numeric_features + cat_names
coefs = final.named_steps['clf'].coef_[0]
```

```
odds = pd.DataFrame({'Feature': all_names, 'Coefficient': coefs})
odds['Odds Ratio'] = np.exp(odds['Coefficient'])
odds = odds.sort_values('Coefficient', key=abs, ascending=False)

print("Top default drivers (by |coefficient|):")
print(odds.head(12).to_string(index=False))

top = odds.head(12).sort_values('Coefficient')
fig, ax = plt.subplots(figsize=(10, 6))
colors = ['crimson' if c > 0 else 'steelblue' for c in top['Coefficient']]
ax.barh(top['Feature'], top['Coefficient'], color=colors)
ax.axvline(0, color='black', linewidth=0.8)
ax.set_xlabel('Log-odds coefficient  (red = raises default risk | blue = lowers␣
 ↪it)')
ax.set_title('Logistic Regression - What Drives Default?', fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("What this means for the Chief Risk Officer: each odds ratio is directly␣
 ↪actionable -")
print("e.g. a higher loan_percent_income (debt burden) and a prior default on␣
 ↪file multiply")
print("the odds of default, while higher income lowers them. Unlike a black-box␣
 ↪model, every")
print("driver here is named, signed, and quantified - exactly the transparency␣
 ↪a lending")
print("decision must withstand from a regulator or a declined applicant.")
```

```
Top default drivers (by |coefficient|):
                   Feature  Coefficient  Odds Ratio
               loan_grade_G     2.101059    8.174825
               loan_grade_B    -1.450732    0.234399
  person_home_ownership_OWN    -1.443782    0.236033
               loan_grade_A    -1.442484    0.236340
       loan_percent_income     1.407562    4.085981
               loan_grade_C    -1.368931    0.254379
               loan_grade_F     0.988245    2.686516
 person_home_ownership_RENT     0.875483    2.400035
               loan_grade_E     0.746775    2.110183
                  loan_amnt    -0.712712    0.490313
       loan_intent_VENTURE    -0.686697    0.503236
loan_intent_HOMEIMPROVEMENT     0.617467    1.854225
```




```
What this means for the Chief Risk Officer: each odds ratio is directly
actionable -
e.g. a higher loan_percent_income (debt burden) and a prior default on file
multiply
the odds of default, while higher income lowers them. Unlike a black-box model,
every
driver here is named, signed, and quantified - exactly the transparency a
lending
decision must withstand from a regulator or a declined applicant.
```

1.7.1 Week 4 Takeaways

What this week established: - The capstone's working target is now **default (`loan_status`)**, a ~1-in-5 minority class, so we evaluate with AUC and precision-recall rather than accuracy, and stratify every split. - **Feature scaling** was shown - not just claimed - to speed convergence and make the L2 penalty fair and coefficients comparable; scaling now lives inside every pipeline. - **Logistic regression** gives a strong, fully transparent baseline whose coefficients read as **odds ratios** a risk officer can defend line by line. - Regularization (L1/L2, swept over C) carries Week 2's insurance logic into classification; performance is robustly flat across a wide C band.

Logistic regression's AUC is the number to beat. Week 6 closes the loop with a head-to-head leaderboard of every classifier built across the capstone.